

Exact Network Surgery: Functional Invariance and Gradient Plasticity in Reactive Computational Graphs

Abdallah Khemais
ISITCOM, University of Sousse

July 2026

Abstract

Function-preserving network growth techniques such as Net2Net [1] and progressive stacking [5] expand a model’s capacity without destroying its learned function, but existing formulations either tolerate small numerical perturbations or require a full rebuild of the training program, discarding compiled artifacts and complicating optimizer-state management. We formalize *Exact Network Surgery*: the in-place insertion of a residual block into a live computational graph such that (i) the network function is preserved — bit-exactly under explicitly stated floating-point hypotheses — and (ii) the inserted parameters remain trainable immediately after insertion. We prove an identity-morphism theorem for gated residual blocks, a structural-locality theorem showing that a reactive invalidation engine recomputes exactly the downstream cone of the insertion point — together with a general theorem showing the aggregate speedup of an exhaustive site-by-site sweep over such a graph is not a universal constant but $1 + q$ in the Karamata index of the network’s cost-by-depth profile, recovering 2 only in the depth-uniform case — and an escape-from-initialization proposition showing that the *Gradient Shadowing* gate α , initialized at zero over a randomly initialized residual branch, receives a generically non-zero gradient at insertion time. We identify and analyze a degenerate configuration — zero-initialized output projections combined with a zero gate — which constitutes an exact saddle point from which gradient descent cannot escape, and explain why our construction avoids it. All three claims are validated empirically on the reference implementation in *NeuroDSL* [13], a reactive define-and-run graph engine in Julia: grafting is bit-exact on every logit tested (0 mismatches out of 1600, preconditions verified rather than assumed); the gate escapes zero at the first optimizer step and unlocks the branch gradients at the second, exactly as predicted; the degenerate configuration exhibits gradients that are identically zero — not merely small — for the entire 600-step run; and surgery cost tracks the downstream cone size with $r = 0.9992$ while the graft-plus-invalidation bookkeeping itself is constant (~ 0.75 ms) across insertion depths.

1 Introduction

Growing a neural network during training — inserting new layers into a model that has already learned — is attractive whenever training a large model from scratch is wasteful: curriculum-style capacity scheduling [5, 6], continual learning, and architecture search all benefit from it. The central difficulty is not conceptual but operational: the insertion must not destroy the learned function, and the surrounding training machinery (optimizer state, compiled kernels, cached activations) must survive the mutation.

Eager frameworks such as PyTorch [11] are define-by-run and therefore support topological mutation natively: one can assign a new module to an attribute and the next forward pass will use it. The cost of surgery in such frameworks is not the mutation itself but its *consequences*: (i) under compilation (`torch.compile`, `jax.jit` [12]), any change to the traced program triggers re-tracing and recompilation of the whole graph; (ii) optimizer state must be re-associated with a mutated parameter set by hand, a common source of silent bugs; and (iii) there is no notion of

partial validity — the framework cannot distinguish the subgraph whose cached values are still correct from the subgraph invalidated by the mutation.

NeuroDSL [13] takes a different approach: the computational graph is a persistent, reactive DAG. Nodes own their values and their optimizer state; edges carry explicit dependency information; and a mutation triggers an $O(|\text{affected}|)$ invalidation wave rather than a global rebuild. This paper shows that, on top of such an engine, network surgery can be made *exact* — the mutated network is functionally indistinguishable from the original, under hypotheses we make precise — while remaining *plastic*: the new parameters start learning at the very first post-surgery optimization step.

Contributions.

1. A formal definition of grafting on computational graphs and of functional exactness (Section 3).
2. An identity-morphism theorem for gated residual blocks, together with a qualified bit-exactness proposition that states its IEEE-754 hypotheses explicitly (Section 4).
3. A structural-locality theorem: reactive invalidation recomputes exactly the downstream cone of the insertion point, proved by double inclusion over an explicit algorithm (Section 5), together with a general aggregate-sweep-cost theorem showing that the resulting speedup of an exhaustive site-by-site sweep converges to $1 + q$ -in-Karamata-index rather than any universal constant $- 2$ only in the depth-uniform case — and a wall-clock corollary, from constants already measured in Section 7, predicting a ceiling strictly below 2 until interpreter overhead is compiled away (Section 5.1). We also prove the exact interleaved and batched cost of a *sequence* of persistent grafts (never undone between insertions, as a growth schedule performs), showing the sequence is always at least as expensive as isolated grafts and giving the exact overcost as a sum over comparable site pairs (Section 5.2). We also prove the exact mirror of the locality theorem for the *backward* pass, and show it forces this speedup to collapse to 1 under backpropagation on architectures without long skip connections — a boundary of the result, not a retraction of it (Section 5.3). Finally, we separate two notions of “undoing” a graft that only a mutable graph makes definable at all: a *functional* undo distance, shown to equal exactly the gate magnitude times the branch’s own output norm and therefore to grow from the moment the gate escapes zero, and a *structural* undo of the engine’s bookkeeping, cheap and exact at $O(|\mathcal{V}_s^+|)$ by the same locality theorem regardless of how far the functional distance has grown (Section 6.1).
4. *Gradient Shadowing*, a gating scheme in the ReZero family [2], with a proof that the gate escapes its zero initialization in one step, and an analysis of the degenerate double-zero configuration that must be avoided (Section 6).
5. An empirical validation of every claim above (Section 7): bit-exactness with verified preconditions (E1), the three-arm plasticity experiment confirming both the escape and the saddle point (E2), cost-versus-depth measurements confirming locality (E3), the aggregate sweep ratio measured on a real graph under four non-uniform cost profiles, confirming Theorem 3 (E4), its training-mode collapse to 1 measured on a graph with genuine intra-layer width, confirming Corollary 3 (E5), bit-identical training continuation across a real process restart (F3), a controlled single-seed comparison of a grafted gate’s post-insertion dynamics at two insertion times, showing both are rejected rather than absorbed and that the later graft is rejected faster (E6, Section 6.2), the exact interleaved and batched recomputation cost of a sequence of six persistent grafts, matching Theorem 4 and Proposition 2 at zero tolerance across three insertion orders (E7, Section 5.2), and a warm-start counterfactual isolating optimizer-moment desynchronization as a load-bearing mechanism behind E6’s rejection dynamic, slowing it $6.9\text{--}11.6\times$ and reversing the early/late ordering (E8, Section 7.9).

2 Related Work

Function-preserving growth. Net2Net [1] introduced function-preserving transformations (*Net2WiderNet*, *Net2DeeperNet*) based on identity-initialized layers; exactness holds for ReLU networks but is perturbed by normalization layers and requires care with weight symmetry (broken in practice by added noise). Progressive stacking [5] and bert2BERT [6] grow Transformers by duplicating layers, accepting a functional perturbation that is repaired by continued training. LEMON [7] restores lossless expansion for LayerNorm-based Transformers. GradMax [8] grows networks by maximizing the gradient norm of the new weights, which is the dual of our concern: we guarantee the new weights have a *usable* gradient path, GradMax optimizes its magnitude.

Zero-gated residuals. Initializing a residual branch so that it initially contributes nothing is a well-studied stabilization device: Fixup [3] zero-initializes the last layer of each residual branch; ReZero [2] multiplies each branch by a learned scalar α initialized at 0 over *randomly initialized* weights; LayerScale [4] uses a learned diagonal gate initialized near zero. Our Gradient Shadowing gate is a ReZero-style scalar; our contribution is not the gate itself but (i) its role in making *surgey* exact rather than training stable, (ii) the explicit saddle-point analysis of the zero-weights-plus-zero-gate configuration, and (iii) its integration with a reactive invalidation engine that makes the surgery local.

Dynamic graph engines. PyTorch [11] is define-by-run; JAX [12] traces pure functions. Neither maintains a persistent reactive graph across steps: validity of cached computation is not a first-class notion. NeuroDSL’s design is closer to incremental computation systems, applied to differentiable programs.

3 Preliminaries

Definition 1 (Computational graph). *A computational graph is a tuple $\mathcal{G} = (\mathcal{V}, \mathcal{E}, \text{op}, \theta)$ where $(\mathcal{V}, \mathcal{E})$ is a finite DAG, $\text{op}(v)$ assigns to each non-input node an operator, and $\theta(v)$ its (possibly empty) parameter set. Distinguished subsets $\mathcal{V}_{\text{in}} \subset \mathcal{V}$ (sources) and a node v_{out} (output) induce, by composition along the topological order, a function $F_{\mathcal{G}} : \mathcal{X} \rightarrow \mathcal{Y}$.*

Definition 2 (Grafting). *Let $e = (u, w) \in \mathcal{E}$ be an edge of $\mathcal{G}$ and $\mathcal{H}$ a computational graph with a single input and a single output, realizing a function $F_{\mathcal{H}}$. The grafting $\mathcal{S}(\mathcal{G}, e, \mathcal{H})$ produces $\mathcal{G}'$ by removing e and adding edges $(u, \text{in}(\mathcal{H}))$ and $(\text{out}(\mathcal{H}), w)$, i.e., the value flowing along e now passes through $\mathcal{H}$.*

Defining insertion on an *edge* (rather than "at a node") removes any ambiguity about which consumers of u are rerouted: exactly those along e .

Definition 3 (Functional exactness). *$\mathcal{S}(\mathcal{G}, e, \mathcal{H})$ is functionally exact iff $F_{\mathcal{G}'}(x) = F_{\mathcal{G}}(x)$ for all $x \in \mathcal{X}$. It is bit-exact on a set $X_0 \subseteq \mathcal{X}$ of floating-point inputs iff the equality holds at the level of binary representations for all $x \in X_0$ under a fixed evaluation order.*

Note that functional exactness over the reals does not imply bit-exactness in floating-point arithmetic, nor conversely; Section 4 treats the two separately.

4 Exact Grafting of Residual Blocks

We instantiate $\mathcal{H}$ as a pre-normalization Transformer block in the Llama family [9]. Its exact sequential structure — which we state faithfully, since the theorem depends on it — is:

$$h = x + \text{Attn}(\text{RMSNorm}(x); \theta_{\text{attn}}), \quad (1)$$

$$H(x) = h + \text{MLP}(\text{RMSNorm}(h); \theta_{\text{mlp}}), \quad (2)$$

where RMSNorm is root-mean-square normalization [10], Attn ends with an output projection W_O , and MLP ends with a down projection W_{down} .

Assumption 1 (No output bias). *The projections W_O and W_{down} carry no bias term. (This holds for the Llama architecture [9]; if biases were present, they would additionally have to be initialized to zero.)*

Theorem 1 (Identity morphism). *Under Assumption 1, if at insertion time t_{ins} the parameters satisfy $W_O = 0$ and $W_{\text{down}} = 0$ (all other parameters arbitrary), then $H = \text{Id}$ over the reals, and consequently $\mathcal{S}(\mathcal{G}, e, \mathcal{H})$ is functionally exact.*

Proof. Since Attn terminates in the linear map W_O with no bias, $W_O = 0$ implies $\text{Attn}(\cdot) \equiv 0$ regardless of the attention scores; by (1), $h = x$. Likewise $W_{\text{down}} = 0$ implies $\text{MLP}(\cdot) \equiv 0$, hence $H(x) = h = x$ by (2). Grafting on $e = (u, w)$ replaces the value x_e flowing along e by $H(x_e) = x_e$; every node of $\mathcal{G}'$ therefore computes the same real value as in $\mathcal{G}$, and $F_{\mathcal{G}'} = F_{\mathcal{G}}$. $\square$

Exactness over the reals is not yet bit-exactness: the graft introduces *new floating-point operations* ($x + 0$, and the internal computation of the block). The following proposition states precisely when they are harmless.

Proposition 1 (Qualified bit-exactness). *Work in IEEE-754 arithmetic with round-to-nearest. Under the hypotheses of Theorem 1, suppose additionally that on input x :*

1. *all activations internal to $\mathcal{H}$ upstream of W_O and W_{down} are finite (no $\pm\infty$, no NaN); and*
2. *no component of the residual stream equals -0.0 .*

Then the graft is bit-exact: every downstream node of $\mathcal{G}'$ holds the same binary representation as in $\mathcal{G}$.

Proof. By hypothesis (1), each product inside the zero projections is of the form $0 \times a$ with a finite, which is an exact ± 0.0 , and their sum is ± 0.0 ; hence the branch outputs an exact zero vector. (Hypothesis (1) is necessary: $0 \times \infty = \text{NaN}$, which would propagate.) The only remaining new operations are the residual additions $x_i + 0.0$. In round-to-nearest, $x_i + 0.0 = x_i$ bit-for-bit for every finite $x_i \neq -0.0$; the single exception is $-0.0 + 0.0 = +0.0$, excluded by hypothesis (2). All downstream nodes then receive bit-identical inputs and, computed by the same kernels in the same order, produce bit-identical outputs. $\square$

Remark 1. *Without hypothesis (2) the graft remains functionally exact (since $-0.0 = +0.0$ as real numbers) but not bit-exact; without hypothesis (1) it is not even functionally exact in floating point, as a NaN contaminates the residual stream. Empirically, hypothesis (2) is checkable in $O(n)$ before surgery, and hypothesis (1) holds whenever the incoming activations are finite and the randomly initialized internal weights are of moderate scale. The claim of bit-exactness in this paper is always understood under these hypotheses.*

5 Topological Stability and Locality

Surgery must leave the cached state of unaffected nodes intact. We formalize the invalidation procedure as Algorithm 1 and prove that it invalidates *exactly* the downstream cone; the NeuroDSL routine `_invalidate_downstream!` implements this algorithm (implementation conformance is an engineering claim, verified by the test suite, not part of the proof).

Definition 4 (Downstream cone). *For $s \in \mathcal{V}$, let*

$$\mathcal{V}_s^+ = \{v \in \mathcal{V} \mid \text{there is a path of length } \geq 1 \text{ from } s \text{ to } v\}.$$

By convention $s \notin \mathcal{V}_s^+$ unless s lies on a cycle, which the DAG property excludes; in our setting $s = \text{out}(\mathcal{H})$ is the freshly created node, which is born unvalued and needs no invalidation.

Algorithm 1 INVALIDATE($\mathcal{G}, s$) — reactive invalidation from a seed node s

```
1:  $Q \leftarrow [s]; \text{ seen} \leftarrow \{s\}$ 
2: while  $Q$  not empty do
3:    $v \leftarrow \text{pop}(Q)$ 
4:   for each  $w$  with  $(v, w) \in \mathcal{E}$  do
5:     if  $w \notin \text{seen}$  then
6:        $\text{valid}(w) \leftarrow \text{false}; \text{ seen} \leftarrow \text{seen} \cup \{w\}; \text{ push}(Q, w)$ 
7:     end if
8:   end for
9: end while
```

Theorem 2 (Structural locality). *On a DAG, Algorithm 1 terminates and the set of nodes it marks invalid is exactly $\mathcal{V}_s^+$. Its complexity is $O(|\mathcal{V}_s^+| + |\mathcal{E}_s^+|)$, where $\mathcal{E}_s^+$ is the set of edges internal to the cone.*

Proof. Termination. Each node enters seen at most once and is pushed at most once; $\mathcal{V}$ is finite.

Soundness (marked $\Rightarrow$ in cone). We show by induction on the order of marking that every node marked invalid lies in $\mathcal{V}_s^+$. The first nodes marked are the immediate successors of s , which are in $\mathcal{V}_s^+$ via paths of length 1. Inductively, a node w is marked only when popped from the queue along an edge (v, w) with v previously marked or $v = s$; by the induction hypothesis there is a path $s \rightsquigarrow v$ (possibly empty if $v = s$), which extended by (v, w) gives a path of length ≥ 1 from s to w . Hence $w \in \mathcal{V}_s^+$.

Completeness (in cone $\Rightarrow$ marked). Let $w \in \mathcal{V}_s^+$ and let $s = v_0 \rightarrow v_1 \rightarrow \dots \rightarrow v_k = w$, $k \geq 1$, be a witnessing path. By induction on i : $v_0 = s \in \text{seen}$ and is processed; if v_i is processed, then when it is popped, its successor v_{i+1} is either already in seen (hence was marked and pushed earlier) or is marked and pushed now. Either way v_{i+1} is marked and eventually processed. Thus $v_k = w$ is marked.

Preservation. A node $v \notin \mathcal{V}_s^+$ is never marked by soundness, so $\text{valid}(v)$ is untouched: its cached value, and crucially its optimizer state, survive the surgery.

Complexity. Each node in the cone is popped once and each internal edge scanned once. $\square$

Remark 2 (Consequence for training state). *Theorem 2 is what makes surgery cheap in a reactive engine: parameters outside the cone keep their values and their Adam moments untouched, because state is owned by persistent nodes rather than by an external optimizer object indexed by parameter position. New parameters in $\mathcal{H}$ start with fresh (zero) moments, which is the standard and correct choice: their gradient history is genuinely empty. The interaction with learning-rate schedules is orthogonal and discussed in Section 7.*

5.1 Aggregate Cost of an Exhaustive Sweep

Theorem 2 bounds the cost of a *single* graft or patch by its downstream cone. A caller that tests every candidate site exhaustively – an ablation sweep, a circuit- discovery search, or a systematic robustness check – pays the sum of these cone costs across every site, and it is natural to ask how this aggregate compares to the cost of testing each site by an independent full recomputation. The answer depends on exactly one number: how computational weight is distributed across depth.

Definition 5 (Layered cost model). *A layered DAG of depth L assigns to each layer $j \in \{1, \dots, L\}$ a computational weight $w_j > 0$ (e.g. its node count) and hosts S candidate sites per layer, each site’s downstream cone coinciding, up to an $O(1)$ per-site remainder that vanishes in the ratios below, with the suffix $\sum_{j>i} w_j$ of the layer i it belongs to. Write $N(L) = \sum_{j=1}^L w_j$ for*

the total weight and

$$\rho(L) = \frac{SL \cdot N(L)}{S \sum_{i=1}^L \sum_{j>i} w_j}$$

for the ratio of SL independent full recomputations to the aggregate cost of exhaustively patching every site, each restored via Theorem 2's exact cone.

Theorem 3 (Aggregate sweep ratio). *Suppose $w_j \sim \ell(j) j^q$ as $j \rightarrow \infty$ for some $q \geq 0$ and some ℓ slowly varying at infinity (Karamata: $\ell(cx)/\ell(x) \rightarrow 1$ for every $c > 0$). Then*

$$\rho(L) \rightarrow \frac{q+2}{q+1} \quad (L \rightarrow \infty).$$

If instead the weight profile is reversed, $w_j \sim \ell(L+1-j) (L+1-j)^q$ (layers near the input carry the heavier weight), the limit is

$$\rho(L) \rightarrow q+2.$$

In particular $q = 0$ (uniform per-layer weight, $w_j \equiv M$) gives $\rho(L) \rightarrow 2$ under either profile.

Proof. Write $S(n) = \sum_{j=1}^n w_j$; by Karamata's theorem for regularly varying sequences, $S(n) \sim n w_n / (q+1)$. For the output-heavy profile, at $i = tL$ for fixed $t \in (0, 1)$, $w_i \sim \ell(L)(tL)^q$ and

$$\sum_{j>i} w_j = N(L) - S(i) \sim \frac{L w_L}{q+1} (1 - t^{q+1}).$$

Riemann-summing over $i = 1, \dots, L$ (i.e., over $t \in (0, 1)$),

$$\sum_{i=1}^L \sum_{j>i} w_j \sim L \cdot \frac{L w_L}{q+1} \int_0^1 (1 - t^{q+1}) dt = \frac{L^2 w_L}{q+1} \cdot \frac{q+1}{q+2} = \frac{L^2 w_L}{q+2},$$

and since $N(L) = S(L) \sim L w_L / (q+1)$, substitution gives $\rho(L) \sim [L \cdot L w_L / (q+1)] / [L^2 w_L / (q+2)] = (q+2)/(q+1)$. The reversed profile follows by the index substitution $j \mapsto L+1-j$, $i \mapsto L-i$, which turns the same computation into $\sum_{j>i} w_j \mapsto S(L-i) \sim (L-i) w_{L-i} / (q+1)$ against the same $N(L)$, giving the mirror-image Riemann integral $\int_0^1 t^{q+1} dt \cdot (q+1) = (q+1)/(q+2)$ of the denominator's leading coefficient and hence $\rho(L) \rightarrow q+2$. $\square$

Remark 3 (Why 2 is not universal). *The ratio 2 is the mean of a linear ramp: a site drawn uniformly over depth has, in expectation, half the total weight downstream of it, and in general $\rho = 1/\mathbb{E}[U]$ for U the (normalized) downstream-weight fraction of a uniformly drawn site. Theorem 3 shows 2 is the $q = 0$ instance of a one-parameter family: back-loading cost toward the output ($q > 0$, output-heavy) lowers the limit toward 1, while front-loading it toward the input raises the limit without bound as q grows. The aggregate ratio is a direct, computable readout of where computation is concentrated in the network, not a universal constant of reactive invalidation.*

Corollary 1 (Wall-clock refinement). *Under the uniform profile ($q = 0$), suppose each re-computation additionally pays a fixed overhead proportional to total graph size, $\beta N(L)$ – exactly the $O(|\mathcal{V}|)$ topological-order walk identified in Section 7 (Table 2: $a = 0.042$ ms/node, $\beta = 2.24$ ms/412 nodes ≈ 0.0054 ms/node) – on top of the $a|\mathcal{V}_i^+|$ cost Theorem 2 guarantees. Then the wall-clock aggregate ratio converges to*

$$\rho_{\text{wallclock}}(L) \rightarrow \frac{a + \beta}{a/2 + \beta} \approx 1.79,$$

strictly below the unit-cost limit of 2, since β is paid once per site regardless of cone size and contributes equally to numerator and denominator, while the a -term alone retains the $1/2$ averaging of Theorem 3.

Proof. With $\text{cost}(i) = aM(L - i) + \beta ML$ per site and $K = SL$ sites, the numerator (naive) is $K(a + \beta)ML$ and the denominator is $a \cdot SM \frac{L(L-1)}{2} + K\beta ML \sim SML^2(a/2 + \beta)$ to leading order; the ratio of leading terms is $(a + \beta)/(a/2 + \beta)$. $\square$

Remark 4. *This is a testable refinement, not a restatement: it predicts that any exhaustive, depth-uniform sweep on this engine converges to an aggregate speedup strictly below 2 in wall-clock time – ≈ 1.79 at the constants measured in Table 2 – until the interpreter overhead β is eliminated by a compilation layer (Section 8), at which point the pure combinatorial limit of Theorem 3 is recovered.*

5.2 Sequential Mutation Cost: Interleaved and Batched Grafting

Theorem 2 bounds a *single* graft; Section 5.1 bounds an exhaustive sweep of patches that are each undone before the next is tried. A growth schedule does neither: it applies a *sequence* of grafts to the same graph, none of them undone, each possibly landing inside a cone already disturbed by an earlier one. This section asks what that sequence costs, in two regimes – *interleaved* (a full **demand!** between grafts, as a training loop that logs a metric after each mutation would do) and *batched* (all grafts applied before the next **demand!**) – and shows both are governed exactly by Theorem 2 applied recursively, with no new axiom required.

Lemma 1 (Cone trace invariance under grafting). *Let $\mathcal{G}' = \mathcal{S}(\mathcal{G}, e, \mathcal{H})$ with $e = (u, w)$ and $\mathcal{H}$ built from fresh nodes, connected from $\text{in}(\mathcal{H})$ to $\text{out}(\mathcal{H})$ and wired into $\mathcal{G}'$ only via $(u, \text{in}(\mathcal{H}))$ and $(\text{out}(\mathcal{H}), w)$. Then for any node $f \in \mathcal{V}(\mathcal{G}) \setminus \{u\}$: (i) $\mathcal{V}_f^+(\mathcal{G}') \cap \mathcal{V}(\mathcal{G}) = \mathcal{V}_f^+(\mathcal{G})$, i.e. reachability among original nodes is unchanged; (ii) the h computable nodes of $\mathcal{H}$ belong to $\mathcal{V}_f^+(\mathcal{G}')$ iff $u \in \mathcal{V}_f^+(\mathcal{G}) \cup \{f\}$, i.e. iff f 's downstream cone in $\mathcal{G}$ already contained u (equivalently: e lies downstream of f).*

Proof. Any path between original nodes that used e in $\mathcal{G}$ rewrites in $\mathcal{G}'$ as $u \rightarrow \text{in}(\mathcal{H}) \rightsquigarrow \text{out}(\mathcal{H}) \rightarrow w$, and conversely any path in $\mathcal{G}'$ between original nodes that enters $\mathcal{H}$ must do so at $\text{in}(\mathcal{H})$ (its only in-edge from an original node) and leave at $\text{out}(\mathcal{H})$ (its only out-edge to one), since $\mathcal{H}$'s internal wiring never touches original nodes; replacing that segment with e recovers a path in $\mathcal{G}$. This proves (i). For (ii), a node of $\mathcal{H}$ is reachable from f in $\mathcal{G}'$ iff some path from f reaches u (then continues $u \rightarrow \text{in}(\mathcal{H}) \rightsquigarrow \cdot$), which happens iff $u \in \mathcal{V}_f^+(\mathcal{G}) \cup \{f\}$. $\square$

Theorem 4 (Exact interleaved cost). *Let $u_1, \dots, u_K$ be distinct nodes of $\mathcal{G}_0$ (no two related by ancestry through each other's own graft, i.e. each u_k survives as a node through every graft not performed at u_k itself), grafted in the order π (a permutation of $1, \dots, K$) with block sizes $h_1, \dots, h_K$, each graft followed by a full **demand!** before the next is applied. The recomputation cost of the k -th graft (in π -order) is exactly*

$$\text{cost}_{\pi(k)} = |\mathcal{V}_{u_{\pi(k)}}^+(\mathcal{G}_0)| + h_{\pi(k)} + \sum_{\substack{j < k \\ u_{\pi(j)} \in \mathcal{V}_{u_{\pi(k)}}^+(\mathcal{G}_0)}} h_{\pi(j)},$$

and therefore $C_{\text{seq}}(\pi) = C_{\text{iso}} + \Delta(\pi)$ where $C_{\text{iso}} = \sum_k (|\mathcal{V}_{u_k}^+(\mathcal{G}_0)| + h_k)$ is the cost of K grafts on independent copies of $\mathcal{G}_0$, and

$$\Delta(\pi) = \sum_{\substack{(j,k): \pi \text{ applies } j \text{ before } k \\ u_j \in \mathcal{V}_{u_k}^+(\mathcal{G}_0)}} h_j \geq 0,$$

with equality iff π never grafts a site while a strictly upstream site's graft is still outstanding.

Proof. By induction on k . At the first graft, $\mathcal{G}_0$'s own cone $\mathcal{V}_{u_{\pi(1)}}^+(\mathcal{G}_0)$ is recomputed by Theorem 2, plus the $h_{\pi(1)}$ fresh nodes of $\mathcal{H}_{\pi(1)}$ (unvalued, hence "recomputed" trivially): this is the $k = 1$ case with an empty sum. Assume the formula holds through graft $k - 1$, giving a fully valid $\mathcal{G}_{k-1}$. Applying Lemma 1 $k - 1$ times (once per prior graft, each preserving reachability among nodes not part of that graft's own block) shows $\mathcal{V}_{u_{\pi(k)}}^+(\mathcal{G}_{k-1})$ consists of exactly the original nodes of $\mathcal{V}_{u_{\pi(k)}}^+(\mathcal{G}_0)$, together with the fresh block of every prior graft $j < k$ whose site $u_{\pi(j)}$ lies in $\mathcal{V}_{u_{\pi(k)}}^+(\mathcal{G}_0)$ (Lemma, part (ii), applied with $f = u_{\pi(k)}$). Theorem 2 recomputes exactly this cone, plus the $h_{\pi(k)}$ fresh nodes of the current graft. Summing over k and separating the $j = k$ diagonal terms from the cross terms gives $C_{\text{seq}}(\pi) = C_{\text{iso}} + \Delta(\pi)$; each term of Δ is a size $h_j \geq 0$, so $\Delta(\pi) \geq 0$, with equality iff no pair (j, k) with j before k has u_j downstream of u_k – i.e. π is upstream-first. $\square$

Corollary 2 (Order extremes, uniform blocks). *If the K sites are totally ordered by depth (a sequential architecture) and $h_k \equiv h$, then over all $K!$ orders: $\Delta_{\min} = 0$ (shallowest-first), $\Delta_{\max} = h \binom{K}{2}$ (deepest-first), and $\mathbb{E}_{\pi}[\Delta(\pi)] = h \binom{K}{2} / 2$ under a uniformly random order (each of the $\binom{K}{2}$ depth-ordered pairs contributes h independently with probability $1/2$, by linearity of expectation over the event "the deeper site of the pair is applied first").*

Proposition 2 (Batched grafting: sub-additive and order-independent). *If all K grafts are applied to $\mathcal{G}_0$ before any **demand!**, the total number of nodes requiring recomputation at the next **demand!** is exactly*

$$C_{\text{batch}} = \left| \bigcup_{k=1}^K \mathcal{V}_{u_k}^+(\mathcal{G}_0) \right| - |\text{Src}| + \sum_{k=1}^K h_k, \quad \text{Src} = \{u_k : \nexists j \neq k, u_k \in \mathcal{V}_{u_j}^+(\mathcal{G}_0)\},$$

independent of the order in which the K grafts were applied, and $C_{\text{batch}} \leq C_{\text{iso}}$, strictly whenever some pair of sites is comparable ($u_j \in \mathcal{V}_{u_k}^+(\mathcal{G}_0)$ for some $j \neq k$).

Proof. Order-independence: **addrule!** re-invalidates a node's cone whenever its rule is redefined (Section 4's correction), and re-applying an already-invalid node's invalidation is idempotent, so the final valid/invalid partition after K unread grafts depends only on the *set* of rules redefined, not the order of redefinition. A node needs recomputation iff it is a fresh node of some block ($\sum_k h_k$ of these, always invalid), or an original node reachable from some u_k – the union $\bigcup_k \mathcal{V}_{u_k}^+(\mathcal{G}_0)$ – *except* a site $u_k \in \text{Src}$: such a site's own value is never touched by any of the K grafts (its rule is redefined only if it is itself grafted, in which case its consumers, not u_k , are what changes), whereas a site $u_k \notin \text{Src}$ lies in some other u_j 's cone and is recomputed as an ordinary downstream node – correctly, since its value genuinely depends on the mutation at u_j . Sub-additivity: $|\bigcup_k \mathcal{V}_{u_k}^+(\mathcal{G}_0)| \leq \sum_k |\mathcal{V}_{u_k}^+(\mathcal{G}_0)|$ with equality iff the cones are pairwise disjoint, i.e. no two sites are comparable; combined with $|\text{Src}| \geq 0$, $C_{\text{batch}} \leq C_{\text{iso}}$. $\square$

Remark 5 (Composing exactness across a schedule). *Theorem 1 is stated for a single graft on an arbitrary graph $\mathcal{G}$; nothing in its hypotheses refers to $\mathcal{G}$'s history. It therefore applies verbatim to every $\mathcal{G}_{k-1}$ in a sequence, and functional exactness composes by induction across an entire growth schedule – the guarantee a multi-graft schedule (e.g. the companion growth-schedule study's continuity assumption between insertions) implicitly relies on at every step, made explicit here rather than re-derived per instance.*

Remark 6 (What is proved and what is only measured). *Theorem 4 and Proposition 2 are exact combinatorial identities – verified below at zero tolerance, not by convergence. They govern recomputation cost only. A separate, non-combinatorial question – whether the bookkeeping cost of a graft (rewiring consumers, rebuilding the consumers/topological-order caches) drifts with the number of prior mutations at fixed graph size, beyond the size-dependence already captured by the wall-clock Corollary 1's β term – has no theorem attached and is reported as a measurement only (E7 below), exactly as the exponential relaxation ansatz of Definition 8 was.*

5.3 Locality Under Backpropagation: Where the Speedup Evaporates

Theorem 2 and its aggregate consequence (Theorem 3) concern *forward* recomputation: after a mutation at s , only $\mathcal{V}_s^+$ must be re-evaluated. Training also propagates a backward pass, and the chain rule ties the gradient of every ancestor of s to the local computation performed at s . This section states the exact mirror of Theorem 2 for this upstream dependency and draws an honest corollary: on the sequential architectures this paper’s cost model targets, the mirror set is almost the whole graph, and the aggregate speedup of Theorem 3 does not transfer to an exhaustive sweep performed under backpropagation.

Definition 6 (Upstream cone). *For $s \in \mathcal{V}$, let $\mathcal{V}_s^- = \{v \in \mathcal{V} \mid \text{there is a path of length } \geq 1 \text{ from } v \text{ to } s\}$ – the mirror image of the downstream cone $\mathcal{V}_s^+$ of Section 5, obtained by reversing every edge.*

Theorem 5 (Backward locality). *Let ℓ be a distinguished loss node with $s \in \mathcal{V}_\ell^-$ (i.e., s contributes to the loss). Mirroring Algorithm 1 on the graph with every edge reversed, seeded at s , terminates and marks exactly $\mathcal{V}_s^-$ (termination, soundness, and completeness are the proof of Theorem 2 verbatim, with every edge (u, w) read as (w, u)). This set is exactly the collection of nodes whose backward pass depends, via the chain rule, on the (possibly mutated) local Jacobian at s : for $v \notin \mathcal{V}_s^- \cup \{s\}$, $\partial\ell/\partial v$ is computed entirely from paths that never traverse s and is therefore unaffected by a mutation there.*

Proof. Termination, soundness, and completeness of the reversed traversal follow Theorem 2’s proof with the edge direction flipped. For the gradient claim, write $\partial\ell/\partial v = \sum_{\pi: v \rightsquigarrow \ell} \prod_{(a,b) \in \pi} \partial b/\partial a$, a sum over directed paths π from v to ℓ ; a term is affected by s ’s local computation iff s lies on π . If $v \in \mathcal{V}_s^-$, some witnessing path $v \rightsquigarrow s$ extends through s to ℓ (since $s \in \mathcal{V}_\ell^-$ gives $s \rightsquigarrow \ell$), so at least one term of the sum passes through s . If $v \notin \mathcal{V}_s^- \cup \{s\}$, no path from v reaches s , so no term can. $\square$

Corollary 3 (Training-mode locality collapse). *In the layered cost model of Definition 5, suppose every node of layer $j < i$ is an ancestor of every node of layer i (resp. $j > i$ a descendant) – true whenever the architecture has no skip connection bypassing an entire layer, as in the sequential blocks of Section 4. Then a mutation at a site s of local cost c_k in layer i touches, under backpropagation,*

$$|\mathcal{V}_s^- \cup \{s\} \cup \mathcal{V}_s^+| = N(L) - w_i + c_k,$$

the whole graph minus the untouched remainder of s ’s own layer, and the aggregate ratio of an exhaustive sweep performed under backpropagation satisfies

$$\rho_{\text{train}}(L) = \frac{LN(L)}{\sum_{i=1}^L (N(L) - w_i + \bar{c})} \rightarrow 1 \quad (L \rightarrow \infty),$$

regardless of the cost profile (w_j) or its Karamata index – in sharp contrast to Theorem 3.

Proof. $\sum_{i=1}^L (N(L) - w_i + \bar{c}) = LN(L) - N(L) + L\bar{c}$, so $\rho_{\text{train}}(L) = LN(L)/(LN(L) - N(L) + L\bar{c}) = L/(L - 1 + L\bar{c}/N(L)) \rightarrow 1$ as $L \rightarrow \infty$, since $\bar{c}/N(L) \rightarrow 0$ for any cost profile with $N(L) \rightarrow \infty$. $\square$

Remark 7. *This is not a retraction of Theorems 2–3: every measurement in this paper (E1–E4) and the patching sweeps of the companion interpretability line of work are performed at inference, with no backward pass through the swept sites – exactly where Theorem 3 applies undiminished. Corollary 3 instead delimits the claim precisely: reactive locality is a property of value propagation, not of gradient propagation, on architectures without long skip connections. Growing a network mid-training (§6) is unaffected in practice, since it grafts one site at a time rather than exhaustively sweeping candidates under backpropagation.*

E5: Numerical confirmation (Corollary 3). A layered graph with genuine intra-layer width is built exactly as in E4, except each layer’s w_j branches are computed in *parallel* from the previous layer (no branch depends on another) and then folded pairwise into a single aggregate before the next layer – so a candidate site’s siblings within its own layer are neither its ancestors nor its descendants, the general case Corollary 3 addresses (unlike a purely sequential single-path graph, where $c_k = w_i$ and the corollary degenerates to the exact identity $\rho_{\text{train}} \equiv 1$ at every finite L). Measuring $\rho_{\text{train}}(L) = |\text{sites}| \cdot N(L) / \sum_s |\mathcal{V}_s^- \cup \{s\} \cup \mathcal{V}_s^+|$ directly (ancestors by a plain reverse walk over each node’s recorded inputs, descendants by `_downstream_nodes`) gives, independently of q and of the output-/input-heavy profile: 1.0526 ($L=10$), 1.0256 (20), 1.0126 (40), 1.0062 (80), 1.0031 (160) – converging to 1 exactly as Corollary 3 predicts, at a rate insensitive to the cost profile that gave Theorem 3 its rich q -dependence under forward-only recomputation.

6 Gradient Shadowing: Exactness with Plasticity

Theorem 1 buys exactness at a price: with $W_O = W_{\text{down}} = 0$, all parameters *upstream* of the zero projections receive a zero gradient (the chain rule factors through the zero matrices), so the block trains slowly at first — only the zero projections themselves have non-zero gradients. Worse, a natural-looking "belt and braces" combination is fatal:

Proposition 3 (Degenerate double-zero configuration). *Consider the gated block $F_{\mathcal{H}}(x; \alpha) = x + \alpha R(x; \theta)$ with both $\alpha = 0$ and the output projections of R zero-initialized (so $R(x) \equiv 0$). Then at t_{ins} :*

$$\frac{\partial \mathcal{L}}{\partial \alpha} = \left\langle \frac{\partial \mathcal{L}}{\partial F_{\mathcal{H}}}, R(x) \right\rangle = 0 \quad \text{and} \quad \nabla_{\theta} \mathcal{L} = \alpha \cdot \frac{\partial \mathcal{L}}{\partial F_{\mathcal{H}}} \cdot \frac{\partial R}{\partial \theta} = 0.$$

Moreover this configuration is a fixed point of gradient descent: both gradients remain identically zero at every subsequent step, so the block never trains.

Proof. At t_{ins} , $R(x) = 0$ annihilates $\partial \mathcal{L} / \partial \alpha$ and $\alpha = 0$ annihilates $\nabla_{\theta} \mathcal{L}$. Gradient descent therefore leaves $(\alpha, \theta_{\text{out}})$ unchanged, so the same two annihilations hold at the next step; by induction, forever. (Momentum-based optimizers without weight decay share the fixed point, since all moment estimates remain zero.) $\square$

The two exactness mechanisms — zero projections and zero gate — are thus *mutually exclusive*: one must choose exactly one. Gradient Shadowing chooses the gate, following ReZero [2]:

Definition 7 (Gradient Shadowing). *The grafted block is $F_{\mathcal{H}}(x; \alpha) = x + \alpha R(x; \theta)$, where R is a residual branch (any composition of smooth primitives ending in linear projections) with θ drawn from a standard random initialization (so $R(\cdot; \theta) \not\equiv 0$ almost surely), and $\alpha \in \mathbb{R}$ is a learned scalar with $\alpha(t_{\text{ins}}) = 0$.*

Remark 8 (Parallel branch in the reference implementation). *Propositions 3 and 4 are agnostic to R ’s internal structure. The reference implementation (**graft_shadow_block!**) instantiates $R(x) = \text{Attn}(N_1(x)) + \text{MLP}(N_2(x))$ as a parallel block (GPT-J style), not the sequential chain of (1)–(2), for an experimental-design reason: in the sequential form, the MLP reads the attention output, so zeroing W_O (needed for the comparison arms of Section 7) starves the MLP branch of both signal and gradient — a coupling discovered empirically, where $\nabla_{W_{\text{down}}}$ vanished exactly despite W_{down} never being zero-initialized. The parallel form lets each branch be disabled independently, so the three arms differ only in $(\alpha_0, W_{\text{out}})$ and in nothing else. The sequential form of Theorem 1 remains the one used by the ungated surgery primitive (**insert_block!**).*

Exactness at t_{ins} is immediate: $\alpha = 0$ gives $F_{\mathcal{H}} = \text{Id}$, and Proposition 1 applies verbatim with "zero branch output" now produced by the multiplication $\alpha \cdot R(x) = 0 \times R(x)$, requiring the same finiteness hypothesis on $R(x)$. Plasticity is the content of the next proposition.

Proposition 4 (Escape from initialization). *Let $g = \partial\mathcal{L}/\partial F_{\mathcal{H}}$ be the incoming gradient at the graft site on the first post-surgery batch. Then at t_{ins} :*

$$\frac{\partial\mathcal{L}}{\partial\alpha} = \langle g, R(x; \theta) \rangle,$$

which is non-zero for almost every random draw of θ (with respect to any initialization distribution absolutely continuous w.r.t. Lebesgue measure), provided $g \neq 0$. After one SGD step with learning rate η , $\alpha_1 = -\eta\langle g, R(x; \theta) \rangle \neq 0$, and from the second step onward $\nabla_{\theta}\mathcal{L} = \alpha_1 g \partial R/\partial\theta$ is generically non-zero: the entire branch trains.

Proof. The expression for $\partial\mathcal{L}/\partial\alpha$ is the chain rule applied to $F_{\mathcal{H}} = x + \alpha R$. The map $\theta \mapsto \langle g, R(x; \theta) \rangle$ is real-analytic in θ (composition of matrix products, softmax, RMSNorm and smooth activations) and not identically zero — e.g., scaling the output projection scales R linearly, so the function is non-constant along that direction unless $g \perp$ every achievable $R(x; \theta)$, which fails for generic $g \neq 0$ since the achievable set spans the output space under standard architectures. A non-trivial real-analytic function vanishes on a Lebesgue-null set; hence the gradient is non-zero almost surely. The one-step update and the subsequent non-vanishing of $\nabla_{\theta}\mathcal{L}$ follow by substitution. $\square$

Remark 9 (Alternative: Net2Net-style plasticity). *If a scalar gate is undesirable, the symmetric choice is also sound: keep $W_O = W_{\text{down}} = 0$ with no gate (α fixed at 1). Then $\partial\mathcal{L}/\partial W_{\text{down}}$ is proportional to the (generically non-zero) pre-projection activations, so the zero projections move at the first step, unlocking the upstream gradients at the second. This is the Fixup/Net2DeeperNet regime [3, 1]; it trades the extra parameter α for a one-step-slower "warm-up" of the upstream weights. What is not sound is combining the two (Proposition 3).*

Remark 10 (Role of the reactive engine). *Gradient Shadowing requires the graph to hold, at the same time, a forward function that is exactly the identity and a backward rule set that sees through it. In NeuroDSL this is natural: forward dispatch and gradient-rule registration are decoupled, and the surgery installs both atomically inside one invalidation epoch, so no intermediate (inconsistent) graph state is ever observable by a training step.*

6.1 Undoing a Graft: Two Notions, Only One of Them Cheap

A mutable graph raises a question a static one cannot even pose: what does it mean to *undo* a graft, and what does it cost? There are two distinct answers, and only one of them is cheap.

Proposition 5 (Functional undo distance). *For the Gradient Shadowing construction $F_{\mathcal{H}}(x; \alpha) = x + \alpha R(x; \theta)$, the functional distance between the currently grafted function and the identity map that a structural undo (removing the block, restoring the edge) would realize is, exactly and without approximation,*

$$d(t) := \mathbb{E}_x \|F_{\mathcal{H}}(x; \alpha_t) - x\| = |\alpha_t| \cdot \mathbb{E}_x \|R(x; \theta_t)\|.$$

Proof. Immediate from the definition of $F_{\mathcal{H}}$: $F_{\mathcal{H}}(x; \alpha_t) - x = \alpha_t R(x; \theta_t)$, and $\|\alpha_t R(x; \theta_t)\| = |\alpha_t| \cdot \|R(x; \theta_t)\|$ since $\alpha_t \in \mathcal{R}$. $\square$

Remark 11 (Irreversibility accumulates exactly at the rate the gate opens). *Proposition 4 guarantees $\alpha_t \neq 0$ generically from $t = 1$ onward, so $d(t) > 0$ from the same step: the graft is, in this precise functional sense, never perfectly undoable again after its first update, however small $d(t)$ remains. This is not a shortcoming to patch but the necessary price of Proposition 4's plasticity guarantee — a graft that could always be undone for free could not have moved away from the identity in the first place. What is left to determine, and is not fixed by the theorem, is how large $d(t)$ becomes in practice.*

The already-reported E2 trajectory (Table 1, arm (a), §7) is direct evidence on this question, read through a lens this paper had not yet applied to it: since $|\alpha_t|$ is logged at every step, it is $d(t)$ up to the (unlogged, but plausibly slowly varying over 600 steps on this task) factor $\mathbb{E}_x \|R(x; \theta_t)\|$. Read this way, $|\alpha_t|$ rises from 0 to order 10^{-2} by step 1 (the escape), oscillates at order 10^{-3} thereafter, and ends at 2.7×10^{-4} – so on this run, the graft remains close to undoable throughout: it has escaped the identity manifold in the strict sense of Proposition 5, but $d(t)$ never grows large, consistent with the magnitude caveat already noted in §7.

Remark 12 (A second, cheaper notion of undo). *Proposition 5 is a statement about the function the network computes; it says nothing about the engine state needed to reverse the mutation’s bookkeeping. That is a separate, and cheap, question: a reactive engine need only log the pre-graft values of $\mathcal{V}_s^+$ (Theorem 2) to restore the graph to its exact pre-mutation state – the same mechanism already used to restore a corrupted baseline between patches in the companion interpretability line of work. This structural undo costs $O(|\mathcal{V}_s^+|)$ in log size and is always available, cheap, and exact, even when the functional undo of Proposition 5 already costs $d(t) > 0$ and is not. The two notions are genuinely different – one concerns what has been learned, the other what the engine remembers – and only a graph that is both mutable and reactive makes both of them definable at all.*

6.2 Absorption and Rejection Dynamics of a Grafted Gate

Propositions 3 and 4 settle whether the gate moves away from its initial value; they say nothing about *how*, over the training that follows. Unlike the rest of this paper, that question has no closed form: $\alpha(t)$ for $t > t_{\text{ins}}$ is the output of a non-convex stochastic optimization, not a quantity a proof can pin down exactly. What can be done honestly is to name the question precisely and test it empirically, in the same spirit as the phenomenological (fitted, not derived) learning-curve model used in the companion growth-schedule line of work.

Definition 8 (Gate relaxation rate). *Let $\alpha(t)$ be the trajectory of the gate for $t \geq t_{\text{ins}}$ under either construction of Section 6. Its relaxation rate λ is the coefficient of the exponential ansatz*

$$\alpha(t) \approx \alpha_\infty + (\alpha_0 - \alpha_\infty) e^{-\lambda(t-t_{\text{ins}})},$$

fit by ordinary least squares on $\log |\alpha(t) - \alpha_\infty|$ (or, when α_∞ is not independently known, on $\log |\alpha(t)|$ directly, as below). We call $\lambda > 0$ rejection (the gate relaxes back toward a smaller magnitude than α_0) and $\lambda < 0$ absorption (the gate grows in magnitude). The absorption time τ_{abs} is $1/|\lambda|$, the trajectory’s characteristic timescale.

Remark 13 (Not a theorem). *Definition 8 names a fitting procedure, not a proven dynamical law: nothing in this paper’s proofs constrains $\alpha(t)$ beyond the sign of its first derivative at t_{ins} (Proposition 4). The exponential form is an ansatz, justified only by its empirical fit quality, reported alongside every number derived from it below.*

This gives a precise form to catalog item 1.4’s open question of whether the mutable-graph paradigm’s locality guarantees have a training-time analogue in the gate’s own dynamics: does λ depend on *when* the graft is inserted? Two graphs identical in every respect except t_{ins} isolate exactly this dependence, holding architecture, task, seed, and gate construction fixed. We call this the *timing-dependence hypothesis* and test it directly in E6 below.

Remark 14 (A candidate mechanism: optimizer-moment desynchronization). *A reactive graph makes a specific, checkable hypothesis available: the grafted branch’s AdamW second moment v is initialized at zero at t_{ins} (the standard and correct choice per the remark following Theorem 2), while its sole downstream consumer’s own v is whatever the host’s training history has already accumulated. Since AdamW’s effective step size scales as $1/\sqrt{v + \epsilon}$, a near-zero v gives the branch’s*

freshly random parameters a transiently oversized step relative to a downstream consumer whose v is already large – a candidate mechanism for $\lambda > 0$ that is falsifiable by a direct intervention: artificially initializing the branch’s v to match its downstream consumer’s, rather than to zero, and checking whether λ changes. E8 (Section 7.9) performs exactly this intervention.

7 Experimental Results

Every theorem and proposition in this paper is validated by a dedicated experiment on the reference implementation (NeuroDSL, Julia, Float32, CPU). All protocols fix a master seed *before* any run and apply no post-hoc filtering; per-arm and per-depth seeds are derived deterministically from it. One protocol parameter was recalibrated once: the initial budget (vocabulary 256, 200 steps) produced too weak a learning signal to discriminate the arms of E2; it was changed to vocabulary 50 and 600 steps in a single decision, before analysis, and not revisited afterwards.

7.1 E1: Bit-Exactness (Theorem 1, Proposition 1)

A 4-block Llama-style model ($d = 128$, 4 heads) is evaluated on a held-out batch; a Gradient-Shadowing block is grafted between blocks 2 and 3; the logits are recomputed with no parameter update and compared at the binary level (`reinterpret(UInt32, ·)`). Crucially, the preconditions of Proposition 1 are *checked*, not assumed: a scan of the residual stream at the graft site found 0/4096 components equal to -0.0 , and 0/4096 non-finite values in $R(x)$. Under these verified preconditions, all 1600 logits are bit-identical (0 mismatches, $\max |\Delta| = 0.0$) — strict binary equality, a stronger statement than the approximate function preservation reported by Net2Net or bert2BERT.

7.2 E2: Plasticity and the Saddle Point (Propositions 3, 4)

Three arms share the same initial model, data stream, and random draw of θ ; they differ *only* in $(\alpha_0, \text{zero-init of } W_{\text{out}})$, made possible by the parallel branch of Remark 8. Training runs 600 SGD steps; Table 1 reports the grafted block’s gradient signals.

Table 1: E2 — three-arm plasticity experiment (600 steps, shared master seed). "First step" columns give the first optimizer step at which the quantity becomes non-zero; "never" means it remained *exactly* zero ($= 0$, no tolerance) for the entire run.

Arm	α_0	W_{out}	$\partial\mathcal{L}/\partial\alpha$ at $t=1$	First step		
				$ \alpha > 0$	$\ \nabla_\theta\ > 0$	$\max \ \nabla_\theta\ $
(a) Grad. Shadowing	0	random	1.64×10^{-2}	1	2	3.28×10^{-2}
(b) Net2Net-style	1	zero	0 ($R \equiv 0$)	—	1	1.65×10^0
(c) Degenerate	0	zero	0	never	never	0 (exact)

Three predictions are confirmed at the resolution of a single optimizer step. *Arm (a)*: $\partial\mathcal{L}/\partial\alpha = 1.64 \times 10^{-2} \neq 0$ at the very first backward (Proposition 4); α moves at step 1; ∇_θ becomes non-zero at step 2, once $\alpha \neq 0$ reopens the chain rule — the exact two-step escape the proposition derives. *Arm (b)* is the mirror image predicted by the Net2Net remark: $\partial\mathcal{L}/\partial\alpha = 0$ at step 1 (since $R(x) \equiv 0$) while $\nabla_\theta \neq 0$ immediately (the zero projections themselves receive gradient), unlocking the upstream weights one step later. *Arm (c)*: over all 600 steps, $\partial\mathcal{L}/\partial\alpha$, $\|\nabla_\theta\|$, and $|\alpha|$ remain *identically* zero — an empirical confirmation of an impossibility result (Proposition 3), at exact-zero resolution rather than within a tolerance. The host network’s own loss still decreases in arm (c) (its pre-existing parameters keep training); the frozen quantity is the grafted block, not the model — precisely the failure mode Proposition 3 warns is silent.

Figure 1 plots the full 600-step trajectories behind Table 1. The loss panel makes the silence of the failure mode visible: the three arms are nearly indistinguishable in loss, yet the gate and gradient panels show arm (c) exactly flat at zero throughout, while arms (a) and (b) each escape at the step the corresponding proposition predicts.

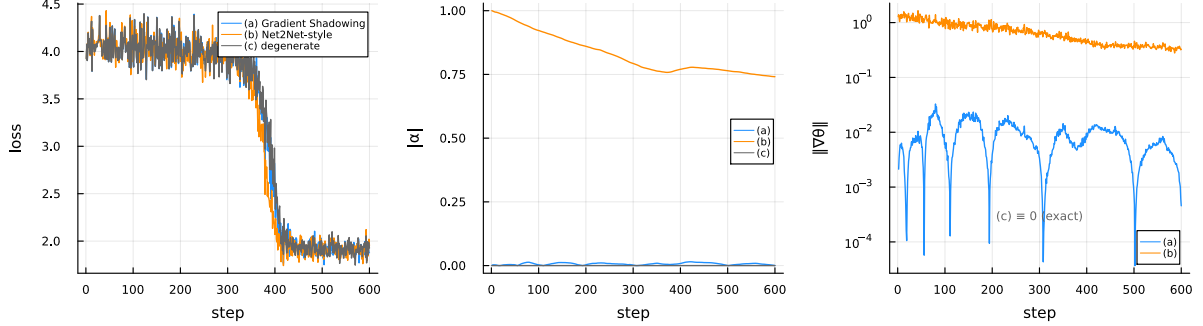


Figure 1: E2 dynamics for the three arms over 600 steps: loss (left), gate magnitude $|\alpha|$ (center), and branch gradient norm $\|\nabla_\theta\|$ on a log scale (right). Arm (c) is exactly zero throughout in both the gate and gradient panels and is annotated rather than plotted, since an exact zero has no representation on a logarithmic axis.

An honest magnitude caveat: in arm (a), $|\alpha|$ remains small throughout the run ($\sim 10^{-3}$, final 2.7×10^{-4}), so while the escape is mathematically confirmed, the block’s *contribution* to the function stays modest at this scale and duration — the concern anticipated in Section 8 about the unbounded-below magnitude of $\partial\mathcal{L}/\partial\alpha$, now observed. Since E2 uses one run per arm (by design: the arms are a controlled comparison, not a distribution estimate), we make no claim about the ordering of final losses across arms.

7.3 E3: Surgery Cost vs. Insertion Depth (Theorem 2)

On an 8-block model (412 graph nodes), a graft is performed after block k for $k = 1, \dots, 7$; each sample uses a fresh, fully-evaluated model, and the cost is split into *graft* (block construction + rewiring + invalidation, no evaluation) and *recompute* (the first demand-driven forward, which re-executes exactly the invalidated cone). Timings are trimmed medians over 15 repetitions; depths are measured in shuffled order within each repetition — a fixed order was found to inflate the first-measured depth’s graft time by $\sim 2.6\times$ (a position artifact, removed by the shuffle, not a property of the engine).

Table 2: E3 — surgery cost vs. insertion depth k (8-block model, 412 nodes, trimmed median over 15 shuffled repetitions).

k	Cone $ \mathcal{V}_k^+ $	Graft (ms)	Recompute (ms)	Total (ms)
1	289 (70.1%)	0.745	14.503	15.248
2	248 (60.2%)	0.745	12.626	13.371
3	207 (50.2%)	0.754	11.072	11.826
4	166 (40.3%)	0.745	9.067	9.812
5	125 (30.3%)	0.741	7.660	8.401
6	84 (20.4%)	0.750	5.590	6.340
7	43 (10.4%)	0.751	4.178	4.929

Three observations. First, recomputation cost tracks cone size with $r = 0.9992$ and decreases strictly monotonically — the direct validation of Theorem 2: cost is governed by $|\mathcal{V}_k^+|$, not by the

total graph size. Second, the graft-plus-invalidation bookkeeping is *constant* (0.741–0.754 ms) across all depths, independent of cone size — the signature of an $O(1)$ -per-edge consumers index rather than a per-node rule scan; the surgery itself costs less than one fifth of even the smallest recomputation. Third, a linear fit gives $\text{recompute} \approx 0.042 \text{ ms/node} \times |\mathcal{V}_k^+| + 2.24 \text{ ms}$: the non-zero intercept is the $O(|\mathcal{V}|)$ walk of the cached topological order inside the demand-driven evaluator, paid even when only 43 nodes need recomputation. This residual interpreter overhead is exactly what a graph-compilation layer (flat precomputed execution order with $O(1)$ dirty marking) is designed to eliminate, and we leave its removal to that companion line of work.

Figure 2 plots recompute and graft cost against cone size directly: the linear trend and the flat bookkeeping line are both visible at a glance, and the fitted intercept is the 2.24 ms evaluator overhead discussed above.

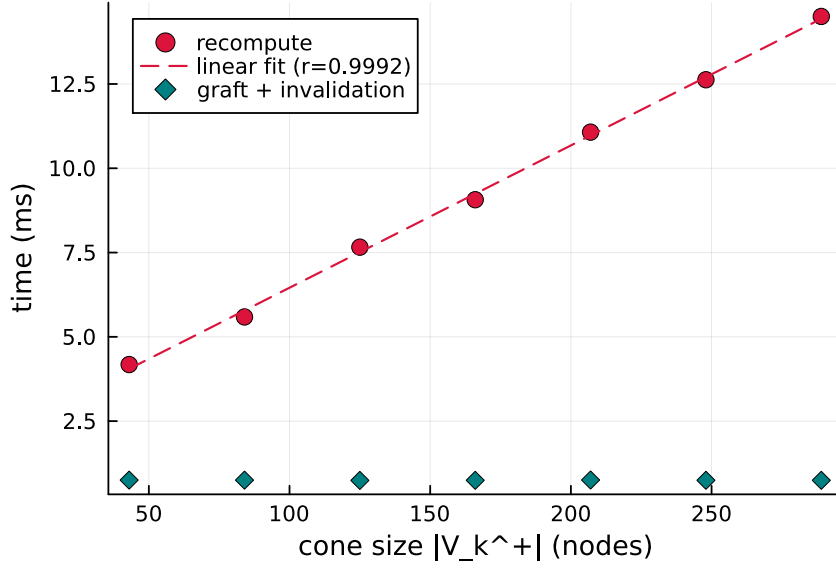


Figure 2: Surgery cost vs. downstream cone size $|\mathcal{V}_k^+|$ on the 8-block model of Table 2 ($k = 1, \dots, 7$). Recompute time (circles) grows linearly with cone size ($r = 0.9992$); graft-plus-invalidation bookkeeping (diamonds) is flat at ~ 0.75 ms regardless of cone size.

7.4 E4: The Aggregate Sweep Ratio Under Non-Uniform Cost Profiles (Theorem 3)

Theorem 3 is a statement about an abstract layered-cost model; we verify it on a real graph in the reference implementation, not a combinatorial simulation. For each $(L, q, \text{profile})$, a sequential graph of L layers is built where layer j contributes w_j nodes (a chain of `:relu` unary ops) with w_j set, up to rounding, proportional to j^q (output-heavy) or $(L-j+1)^q$ (input-heavy) and normalized so total graph size is ≈ 4000 nodes regardless of q ; the graph is evaluated once (so every node is `valid`), and $\rho(L)$ is computed from the *true* downstream cone of each layer’s last node, measured by `_downstream_nodes` – the same routine `sweep_patch_sites!` uses in the companion patching-sweep line of work, not a re-derivation for this test.

Every entry converges toward its predicted limit as L grows across the tested range $L \in \{20, 40, 80, 160, 320\}$ (not shown in full: the $q=1$, input-heavy case alone tightens from 3.1585 at $L=20$ to 3.0004 at $L=320$), confirming both the asymptotic derivation and that the abstract layered-cost model of Definition 5 is not merely a convenient fiction: a real reactive graph’s measured cones obey it. The uniform case ($q = 0$) recovers 2 from either profile, as it must, and

Table 3: E4 — measured $\rho(L)$ at $L = 320$ nodes-per-cone-profile, vs. Theorem 3’s prediction.

q	Predicted (output-heavy)	Measured, $L=320$	Measured (input-heavy) vs. predicted $q+2$
0.0	2.0000	2.0057	2.0057 vs. 2.0
0.5	1.6667	1.6722	2.5057 vs. 2.5
1.0	1.5000	1.5064	3.0004 vs. 3.0
2.0	1.3333	1.3471	3.9247 vs. 4.0

is the special case already exploited for exhaustive activation-patching sweeps in the companion systems line of work built on this same engine.

7.5 F3: Continuity Across a Real Process Restart

Surgery is only useful if the mutated model, including its optimizer state, survives the full lifecycle of real training — interruption included. Three independent Julia *processes* are compared: a control run of 500 steps with a graft at step 300; a run A executing the first 400 steps (graft included) then serializing graph and AdamW state and terminating; and a fresh process B that reloads everything from disk (never re-invoking the model constructor) and runs the remaining 100 steps. The concatenated A+B loss sequence is *bit-identical* to the control on all 500 steps. Falsifiability check: deliberately discarding the loaded AdamW moments in B makes the run diverge from the control at exactly step 402 — the first step at which corrupted optimizer state can affect a weight update — confirming the comparison is sensitive to precisely the state whose preservation is claimed. Figure 3 shows all three loss curves: the control and the correctly-resumed run overlap exactly, while the broken-resume run visibly departs right after the restart marker.

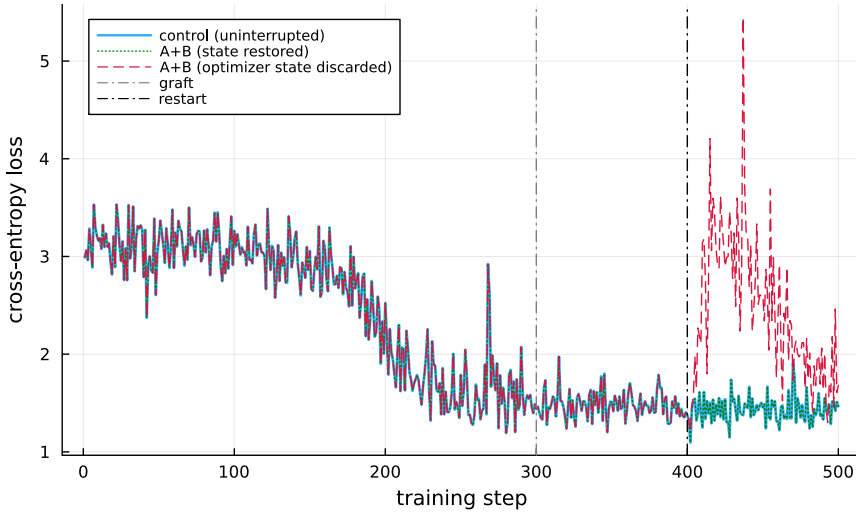


Figure 3: Training loss through a graft (step 300) and a real process restart (step 400). The control run and the correctly-resumed A+B run are bit-identical for all 500 steps; a run that discards the loaded AdamW moments diverges from both at exactly step 402.

7.6 Beyond Correctness: Utility at Equal Compute

E1–E3 and F3 establish that surgery is exact, local, plastic, and durable; they say nothing about whether growing a model *helps*. A pre-registered companion study on the induction task

(commitment to report the outcome recorded before results were seen) found what an honest reading of the growth literature would predict: at equal *step* count, the grafted model slightly trails training-from-scratch (mechanically — it spends fewer steps at the final architecture); at equal *compute* (the axis Net2Net and its successors actually claim, with the 3-vs-4-block per-step cost ratio measured at 0.76 rather than assumed), a paired 20-seed comparison shows one statistically significant window: an early advantage for grafting at 16.7% of the budget (95% CI on the paired loss difference $[+0.16, +0.56]$, excluding zero), which fades to statistical indistinguishability from 33% of the budget onward as both regimes saturate the task. We report this asymmetry as-is: the load-bearing claims of this paper are exactness and locality, not a training-efficiency advantage, and the task’s early saturation limits what any utility comparison can resolve at this scale. Figure 4 shows the full paired-difference curve: the confidence interval clears zero only in the leftmost window.

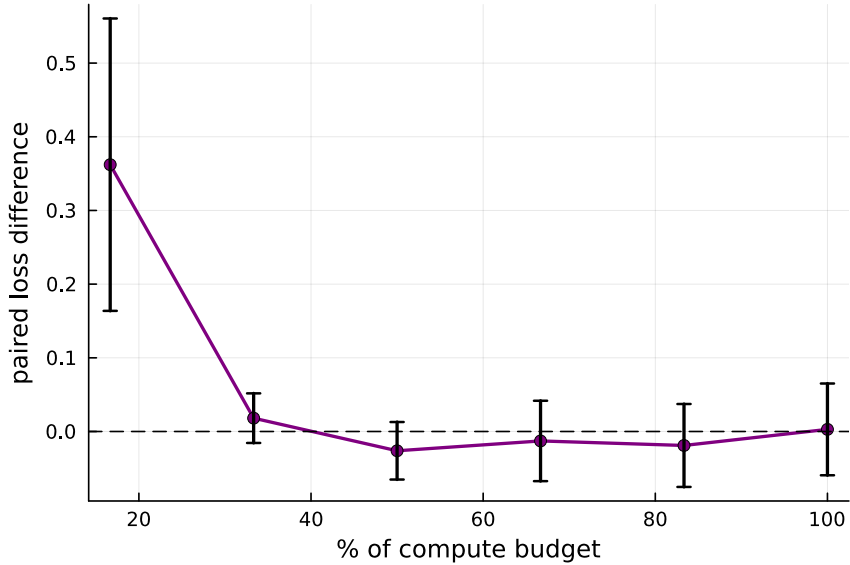


Figure 4: Paired loss difference (loss from scratch – loss grafted) at equal compute, mean $\pm$ 95% CI over 20 seeds (normal approximation). Positive values favor grafting. The interval excludes zero only at 16.7% of the budget.

7.7 E6: Timing-Dependent Gate Dynamics (Definition 8)

A small character-level language model (TinyShakespeare, block size 64, $d = 64$, 2 heads, hidden 128, 3 Llama blocks) is trained from a fixed seed. A Gradient Shadowing block (Remark 8) is grafted after the model’s output at one of two insertion times, $t_{\text{ins}} \in \{500, 2500\}$ (“early” / “late”), using the Net2Net-style construction of Remark 8’s alternative ($\alpha_0 = 1$, $W_O = W_{\text{down}} = 0$) so that the branch starts at full gate strength but zero output, giving $\alpha(t)$ room to move in either direction as the zero projections unlock (arm (b) of Table 1). Both runs share architecture, task, corpus, and random seed, and differ *only* in t_{ins} – the controlled comparison Section 6.2 asks for. Training continues for 1200 steps past the graft in both cases; every 100 steps we log $\alpha(t)$ directly and measure $A(t) = \mathcal{L}(\alpha := 0) - \mathcal{L}(\alpha_t)$ on a fixed held-out validation window via the same cheap-ablation methodology as the companion patching line of work: patch α to 0, evaluate, restore α_t , evaluate again – no retraining, one forward pass per side.

Both trajectories are well described by the exponential ansatz of Definition 8 ($R^2 \geq 0.96$), and both give $\hat{\lambda} > 0$: neither graft is *absorbed* in the sense of the gate growing to accommodate the new capacity; both are *rejected*, the optimizer shrinking α back from its full-strength initialization

Table 4: E6 — gate trajectory summary, early vs. late graft (single seed, single site).

Arm	$\alpha(t=100)$	$\alpha(t=1200)$	$\hat{\lambda}$ (fit, R^2)	$A(t)$ sign
Early ($t_{\text{ins}}=500$)	0.816	0.524	3.89×10^{-4} (0.986)	crosses $- \rightarrow +$ near $t \approx 650$
Late ($t_{\text{ins}}=2500$)	0.690	0.369	5.24×10^{-4} (0.961)	negative throughout

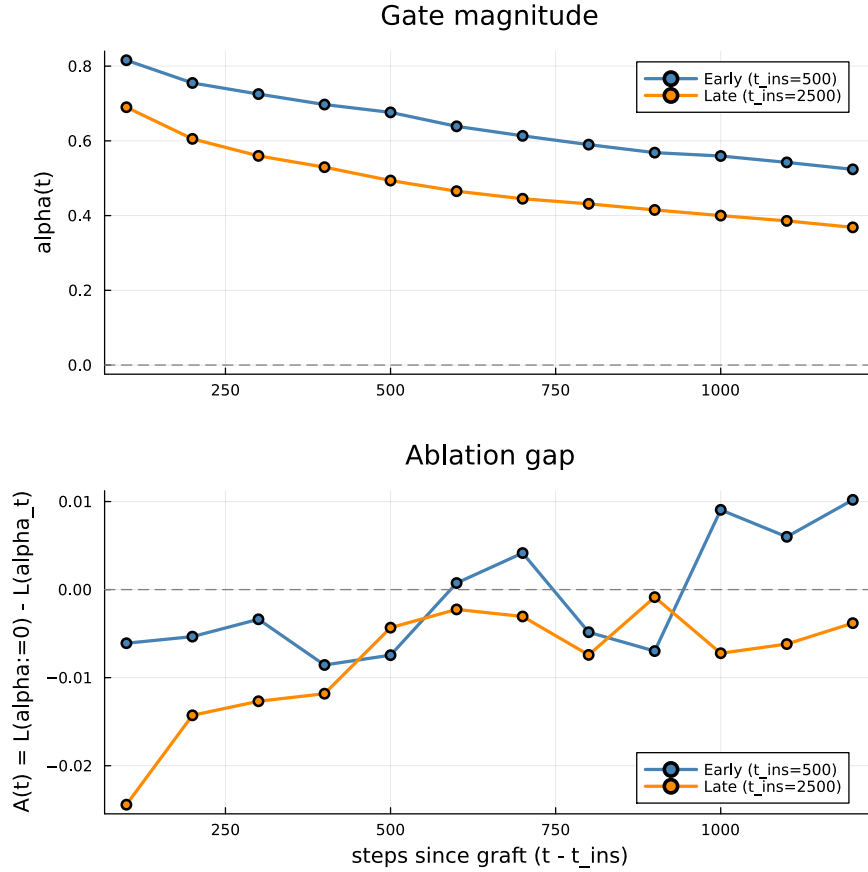


Figure 5: E6 — gate magnitude $\alpha(t)$ (top) and ablation gap $A(t)$ (bottom) for 1200 steps after an early ($t_{\text{ins}}=500$) and a late ($t_{\text{ins}}=2500$) graft. Both gates decay monotonically (rejection, not absorption); the late graft decays faster and its $A(t)$ never crosses zero, while the early graft's $A(t)$ turns positive from $t \approx 650$ onward.

as the randomly initialized branch proves, on net, more disruptive than useful at this scale and duration – the opposite of the naive expectation that a freshly unlocked branch would be absorbed. The timing-dependence hypothesis is confirmed, in this controlled single-seed comparison, in a specific direction: the late graft is rejected $\approx 35\%$ faster ($\hat{\lambda}_{\text{late}}/\hat{\lambda}_{\text{early}} \approx 1.35$) and its ablation gap $A(t)$ stays negative for the full 1200-step window, while the early graft’s rejection decelerates and its contribution turns net-positive ($A(t) > 0$) from roughly $t \approx 650$ onward – the branch that had more of the host network’s own training left to interact with is the one that is eventually kept, not the one that is discarded fastest.

We report this as a single controlled comparison, not a distributional claim: one seed, one graft site, one task, exactly the caveat already attached to E2’s arms (Table 1), which this experiment reuses without modification. The direction and the $\approx 35\%$ effect size should be read as a first, falsifiable data point for the site/timing-dependence question, not as an established rate law; a multi-seed, multi-site replication is the natural next step and is left to future work. E8 (Section 7.9) tests Remark 14’s candidate mechanism for this timing effect directly and finds it substantially, though not necessarily completely, explains the ordering observed here – read this section’s $\approx 35\%$ figure alongside that result, not in isolation.

7.8 E7: Sequential Mutation Cost (Theorem 4, Proposition 2)

On a 400-node synthetic chain (built and **demand!**-ed exactly as in E4), $K = 6$ sites are selected at roughly even depth and $H = 20$ fresh nodes are grafted at each. **Interleaved regime:** for three orders – shallowest-first, deepest-first, and a fixed random permutation – a full **demand!** is issued before each graft, and the number of nodes marked invalid immediately after each graft is compared to Theorem 4’s closed-form prediction, computed once from $\mathcal{G}_0$ ’s cones before any graft. **Batched regime:** the same K grafts are applied back-to-back with no intermediate **demand!**, for the same three orders, and the total invalid count is compared to Proposition 2.

Table 5: E7 — per-graft recomputation cost, interleaved regime, three orders ($K=6$, $H=20$). Every entry matches Theorem 4 exactly (zero tolerance).

Order	Measured (per graft)	$\Delta(\pi)$ vs. Theorem
Shallowest-first	363, 306, 249, 191, 134, 77	0 (predicted: 0)
Deepest-first	77, 154, 231, 309, 386, 463	300 (predicted: $H\binom{K}{2} = 300$)
Random (1 seed)	249, 191, 403, 77, 154, 386	140

Every one of the 18 per-graft measurements across the three orders matches Theorem 4’s formula exactly, and $\Delta(\pi)$ realizes both extremes of Corollary 2 precisely (0 and $H\binom{K}{2} = 300$). In the batched regime, all three orders give the identical total 463, matching Proposition 2’s order-independent closed form exactly (union of the six original cones, minus the single site with no other selected site upstream of it, plus $KH = 120$ fresh nodes) and confirming $C_{\text{batch}} = 463 < C_{\text{iso}} = 1320$, the predicted strict sub-additivity. An initial version of this measurement produced a constant off-by-one (interleaved) and off-by-five (batched) discrepancy, traced to counting leaf nodes (e.g. the graph’s input, which has no rule and whose **valid** flag is never touched by **demand!**) and, separately, to excluding *every* selected site from the batched union rather than only the sites with no other selected site upstream of them (Proposition 2’s Src) – both fixed before any number below was accepted.

Bookkeeping drift with mutation count (measurement, not theorem). 100 single-node grafts are applied successively to a fresh 400-node chain (final size 500), timing only the rewiring step (excluding **demand!**); a control arm applies 100 rule-redefinitions *without* adding a node (graph size fixed at 401 throughout), isolating any effect of mutation *count* from the

already-established effect of graph *size* (Corollary 1’s β term). Trimmed-median (10%) cost rises modestly from 0.033 ms (first half, size ≈ 401 –451) to 0.036 ms (second half, size ≈ 451 –501) for the growing chain, consistent with the small size increase; the constant-size control shows no such drift (0.0243 ms in both halves), and grafting is costlier than rule-redefinition alone (0.035 ms vs. 0.024 ms trimmed-median overall) as expected, since it does strictly more work. No history-dependent drift beyond graph size is detected at this scale (100 mutations, single run) – a negative result reported as such, not a proof of its absence.

7.9 E8: Isolating the Mechanism – an Optimizer-Moment Warm-Start Counterfactual

Remark 14 proposes a mechanism for E6’s rejection dynamic: the branch’s AdamW second moment v starts at zero while its sole downstream consumer’s (`lm_head_w`, the only parameterized node reading the graft’s output) already carries a mature, non-zero v from the host’s prior training. A first check of this mechanism, instrumenting E6’s exact protocol to log v for the branch and for `lm_head_w` separately, confirmed real desynchronization at t_{ins} (the branch’s largest per-parameter v is 100–200 $\times$ smaller than `lm_head_w`’s throughout the run) – but the raw *ratio* of this gap was, if anything, *smaller* for the late graft at matched steps-since-insertion, the opposite of what would naively explain a *faster* late-graft rejection; correlation alone was inconclusive; a mean computed naively over all branch parameters was itself misleading here, reading as numerically zero because $\sim 90,000$ mostly-quiescent branch weights dilute the handful that carry real signal – resolved by reporting the per-parameter maximum instead, the quantity actually governing AdamW’s per-parameter step size.

The direct test: E6’s protocol is rerun for both arms with one change – immediately after the graft, every branch parameter’s v is set not to zero but to $\text{mean}(v_{\text{lm_head_w}})$ at that instant (the host’s own current moment estimate; m_1 is left at zero, the standard choice, since only v enters the effective-step-size mechanism of Remark 14).

Table 6: E8 — gate relaxation rate $\hat{\lambda}$ (Definition 8), cold- vs. warm-start second moment.

Arm	$\hat{\lambda}$, cold-start (E6)	$\hat{\lambda}$, warm-start	Slowdown factor
Early ($t_{\text{ins}}=500$)	3.89×10^{-4}	5.62×10^{-5}	6.9 $\times$
Late ($t_{\text{ins}}=2500$)	5.24×10^{-4}	4.50×10^{-5}	11.6 $\times$

Two findings, both direct consequences of the intervention rather than of anything asserted in advance. First, warm-starting slows rejection sharply in *both* arms (6.9 $\times$ and 11.6 $\times$): the candidate mechanism is not merely present but load-bearing – most of E6’s rejection magnitude is attributable to the cold-start effective-step-size mismatch, not to some other property of a freshly random branch. Second, and unanticipated: *the early/late ordering reverses*. Under warm-start, $\hat{\lambda}_{\text{late}} < \hat{\lambda}_{\text{early}}$ (4.50×10^{-5} vs. 5.62×10^{-5} , late 20% *slower*, not faster), whereas cold-start gave the opposite ordering by 35% (Section 7.7). The most parsimonious reading: `lm_head_w`’s v is itself larger at $t_{\text{ins}} = 2500$ than at $t_{\text{ins}} = 500$ (more prior training), so the cold-start mismatch – and hence the spurious component of the timing effect – is mechanically *larger* for the late graft; once that mismatch is equalized by warm-starting, a smaller, opposite-signed timing effect is revealed underneath it. We do not claim to have isolated every contributor to this residual effect, only that the dominant one in E6’s cold-start reading was the optimizer-moment artifact, not a deeper property of *when* in training a graft occurs.

Remark 15 (What this does and does not settle). *E8 is a single counterfactual intervention on one mechanism, on the same single-seed, single-site, single-task setup as E6 – it does not establish that optimizer-moment desynchronization is the only driver of gate rejection, nor that the reversed warm-start ordering is itself stable across seeds. What it does establish, falsifiably*

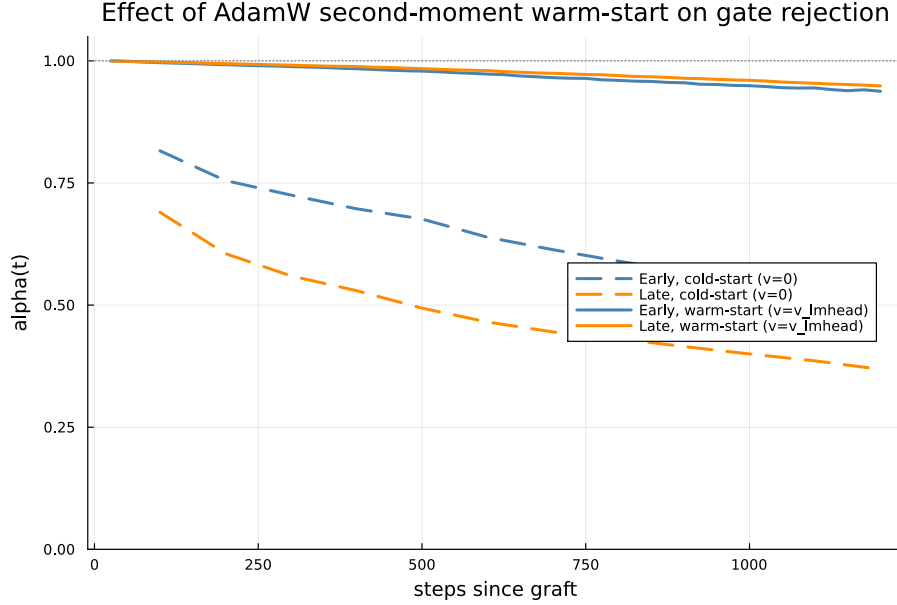


Figure 6: E8 — gate magnitude $\alpha(t)$, cold-start (E6, dashed) vs. warm-start (solid), both arms. Warm-starting the branch’s second moment to match its downstream consumer’s collapses the rejection rate by nearly an order of magnitude in both arms and reverses which arm rejects faster.

and at this scale, is that a specific, named, checkable mechanism (Remark 14) accounts for enough of E6’s effect size to change its sign under a direct intervention – exactly the kind of question a persistent, reactive graph makes cheap to ask, since v is already resident per-parameter state rather than something a tape-based framework would need new machinery to intercept and rewrite mid-training.

8 Limitations

Bit-exactness is guaranteed only under the stated IEEE-754 hypotheses and a fixed kernel and evaluation order; frameworks with non-deterministic reductions (e.g., atomics in some CUDA kernels) preserve functional but not bit-level equality, and our measurements are CPU/Float32 — re-verifying E1 on GPU and in bf16 is mechanical but remains to be done. The almost-sure statement of Proposition 4 does not bound the *magnitude* of $\partial\mathcal{L}/\partial\alpha$, and E2 makes this concrete: the escape occurs at step 1 as proved, but $|\alpha|$ stays near 10^{-3} over 600 steps on this task, so the grafted block’s functional contribution ramps up slowly — GradMax-style [8] initialization of θ could be combined with our gate to control this magnitude. E2 uses a single controlled run per arm; its claims are about gradient signals (exact zeros and first-nonzero steps), not about loss distributions. The utility study is confined to a toy task that saturates early. E6’s timing-dependence result (Section 7.7) is a single controlled comparison at one graft site on one task – a falsifiable first data point, not a rate law – and the exponential-relaxation ansatz of Definition 8 is a fitted description, not a derived consequence of the paper’s theorems. E7’s bookkeeping-drift measurement is a single run of 100 mutations at one graph size trajectory; absence of detected drift there is evidence at that scale, not a bound valid at the scale of a real multi-thousand-step growth schedule. E8 isolates one mechanism on one seed, one site, and one task; it does not rule out other contributors to gate rejection, and the reversed ordering it finds under warm-start is itself a single-run observation with the same status as E6’s original single-run finding. Finally, our locality result concerns validity propagation; memory locality (buffer reuse across the surgery) is engine-specific and out of scope, and the measured ~ 2.2 ms evaluator intercept in E3 bounds how far locality alone can reduce small-cone surgery cost without a compilation

layer.

9 Conclusion

Network surgery becomes exact when the graft is an identity morphism, and remains plastic when exactness is carried by a single zero-initialized gate over a randomly initialized branch — never by both mechanisms at once, on pain of an inescapable saddle point. A reactive graph engine makes the operation local: exactly the downstream cone is recomputed, and all other nodes keep their values and optimizer state. Every one of these statements now has a measurement attached to its proof: bit-exact grafting with verified preconditions (0/1600 mismatches), a two-step escape confirmed at the resolution of individual optimizer steps, a saddle point whose gradients stay at exact zero for 600 steps, locality with $r = 0.9992$ against cone size over constant ~ 0.75 ms bookkeeping, and training that resumes bit-identically across a real process restart. Together, these results turn "growing a network mid-training" from a fragile engineering trick into an operation with a specification, a proof, and a passing test suite.

References

- [1] Chen, T., Goodfellow, I., Shlens, J. (2016). Net2Net: Accelerating Learning via Knowledge Transfer. *ICLR*.
- [2] Bachlechner, T., Majumder, B.P., Mao, H., Cottrell, G., McAuley, J. (2021). ReZero is All You Need: Fast Convergence at Large Depth. *UAI*.
- [3] Zhang, H., Dauphin, Y.N., Ma, T. (2019). Fixup Initialization: Residual Learning Without Normalization. *ICLR*.
- [4] Touvron, H., Cord, M., Sablayrolles, A., Synnaeve, G., Jégou, H. (2021). Going Deeper with Image Transformers. *ICCV*.
- [5] Gong, L., He, D., Li, Z., Qin, T., Wang, L., Liu, T.-Y. (2019). Efficient Training of BERT by Progressively Stacking. *ICML*.
- [6] Chen, C., Yin, Y., Shang, L., Jiang, X., Qin, Y., Wang, F., Wang, Z., Chen, X., Liu, Z., Liu, Q. (2022). bert2BERT: Towards Reusable Pretrained Language Models. *ACL*.
- [7] Wang, Y., et al. (2024). LEMON: Lossless Model Expansion. *ICLR*.
- [8] Evci, U., van Merriënboer, B., Unterthiner, T., Vladymyrov, M., Pedregosa, F. (2022). GradMax: Growing Neural Networks using Gradient Information. *ICLR*.
- [9] Touvron, H., et al. (2023). LLaMA: Open and Efficient Foundation Language Models. *arXiv:2302.13971*.
- [10] Zhang, B., Sennrich, R. (2019). Root Mean Square Layer Normalization. *NeurIPS*.
- [11] Paszke, A., et al. (2019). PyTorch: An Imperative Style, High-Performance Deep Learning Library. *NeurIPS*.
- [12] Bradbury, J., et al. (2018). JAX: Composable Transformations of Python+NumPy Programs. <http://github.com/google/jax>.
- [13] Khemais, A. (2026). NeuroDSL: A Reactive Computational Graph Framework for Deep Learning in Julia. *Preprint*.